

Main Question Example: Thompson at Finite Budget vs High Reference

This document is a single, self-contained research workflow focused on one question:

with a finite rollout budget, does Thompson sampling allocate simulation effort in a statistically sensible way, and how close does it get to a stronger high-budget reference estimate?

It is centered on these functions:

- `bg_initial_board()`
- `bg_roll()`
- `generate_legal_moves()`
- `evaluate_actions_thompson()`
- `approximate_action_reference()`
- `compare_thompson_to_reference()`
- `study_budget_tradeoff()`
- `study_variance_controls()`
- `summarize_thompson_benchmark()`

Additional helper functions are included where they improve interpretation:

- `bg_print_board()`, `bg_plot_board()`
- `compare_action_posteriors()`, `explain_move_evaluation()`
- `trace_thompson_allocation()`, `plot_thompson_convergence()`
- `compare_methods_on_position()`
- `bg_plot_budget_stability()`
- `benchmark_thompson()`, `plot_thompson_vs_baselines()`

1. Statistical framing before code

Fix one local decision instance $d = (s, r)$ with board state s and realized roll r . Let legal actions be $A_d = \{1, \dots, K\}$. For each action i , rollout reward $Y_i \in [0, 1]$ has mean

$$\mu_i = \mathbb{E}[Y_i].$$

The rollout-model best action is

$$a^* = \arg \max_i \mu_i.$$

A finite-budget method uses total budget N and returns recommendation $\hat{a}_N$. Core evaluation quantities are:

- probability of correct selection (PCS):

$$\Pr(\hat{a}_N = a^*)$$

- simple regret:

$$\mu_{a^*} - \mu_{\hat{a}_N}$$

- value-estimation error, often summarized by MSE.

In this package, we approximate a^* using a high-budget reference estimate, not exact truth.

2. Thompson intuition in this package

Thompson sampling maintains posterior uncertainty for each action value, samples one draw per action, and allocates the next rollout to the sampled leader.

Conceptually:

1. maintain posterior for each μ_i ,
2. draw $\tilde{\mu}_i$ from each posterior,
3. allocate next rollout to $\arg \max_i \tilde{\mu}_i$,
4. update posterior for sampled action.

Why this matters here:

- clearly weak actions should lose allocation over time,
- plausible leaders should keep receiving budget,
- hard near-tie states should show slower stabilization.

3. Set one fixed decision instance and budgets

Approx runtime guidance (laptop-class machine):

- quick run: `reference_budget = 12000` and smaller budget grids,
- deep run: `reference_budget = 30000` and larger budget grids.

```
library(backgammonr)

board <- bg_initial_board()
roll <- bg_roll(1L, 6L)

rollout_policy <- "random"
max_rollout_turns <- if (isTRUE(getOption("bg_quick_mode", TRUE))) 250L else 300L
seed <- 11L

finite_budget <- if (isTRUE(getOption("bg_quick_mode", TRUE))) 1200L else 2000L
reference_budget <- if (isTRUE(getOption("bg_quick_mode", TRUE))) 12000L else 30000L

budget_grid <- if (isTRUE(getOption("bg_quick_mode", TRUE))) {
  c(128L, 256L, 512L, 1024L, 2048L)
} else {
  c(128L, 256L, 512L, 1024L, 2048L, 4096L)
```

```

}

benchmark_budgets <- if (isTRUE(getOption("bg_quick_mode", TRUE))) c(256L, 1024L) else c(256L, 1024L, 4096L)
trace_every <- if (isTRUE(getOption("bg_quick_mode", TRUE))) 120L else 80L

```

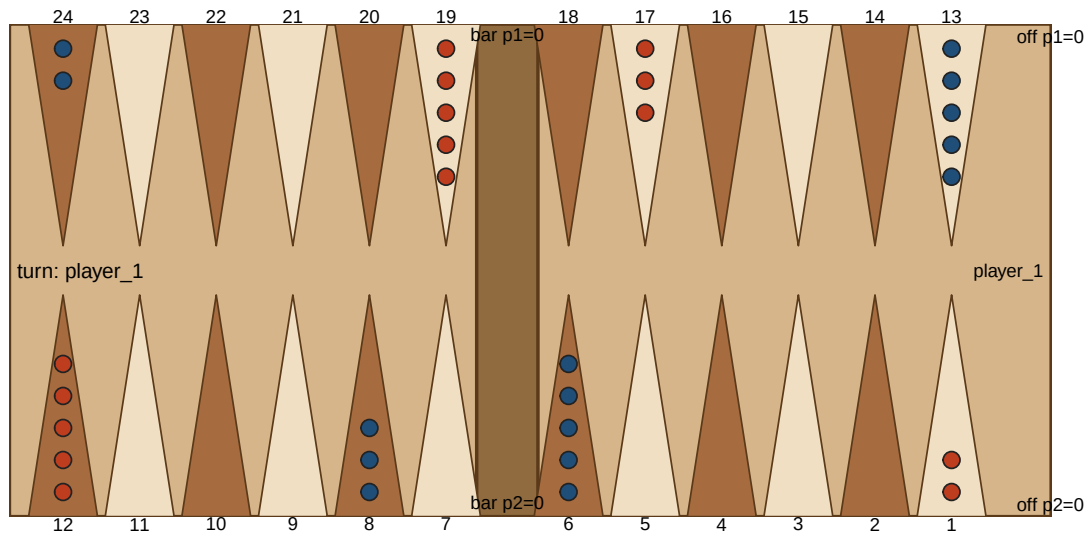
Interpretation: all comparisons below are conditioned on the same board, roll, rollout policy, and horizon.

4. Print and plot the board and roll

```

bg_print_board(board)
#> <bg_board_ascii>
#> turn: player_1
#> perspective: player_1
#> bar: p1=0 p2=0
#> off: p1=0 p2=0
#>
#> Top (24 -> 13)
#>    24    23    22    21    20    19    18    17    16    15    14    13
#>    X2     .     .     .     .    05     .    03     .     .     X5
#> Bottom (12 -> 1)
#>    12    11    10     9     8     7     6     5     4     3     2     1
#>    05     .     .     .    X3     .    X5     .     .     .     .    02
print(roll)
#> <bg_roll>
#> dice:    1 6
#> double: no
#> expanded: 1 6
bg_plot_board(board)

```



Use both:

- `bg_print_board()` for exact textual state,
- `bg_plot_board()` for quick visual orientation.

5. Generate legal actions and inspect action space

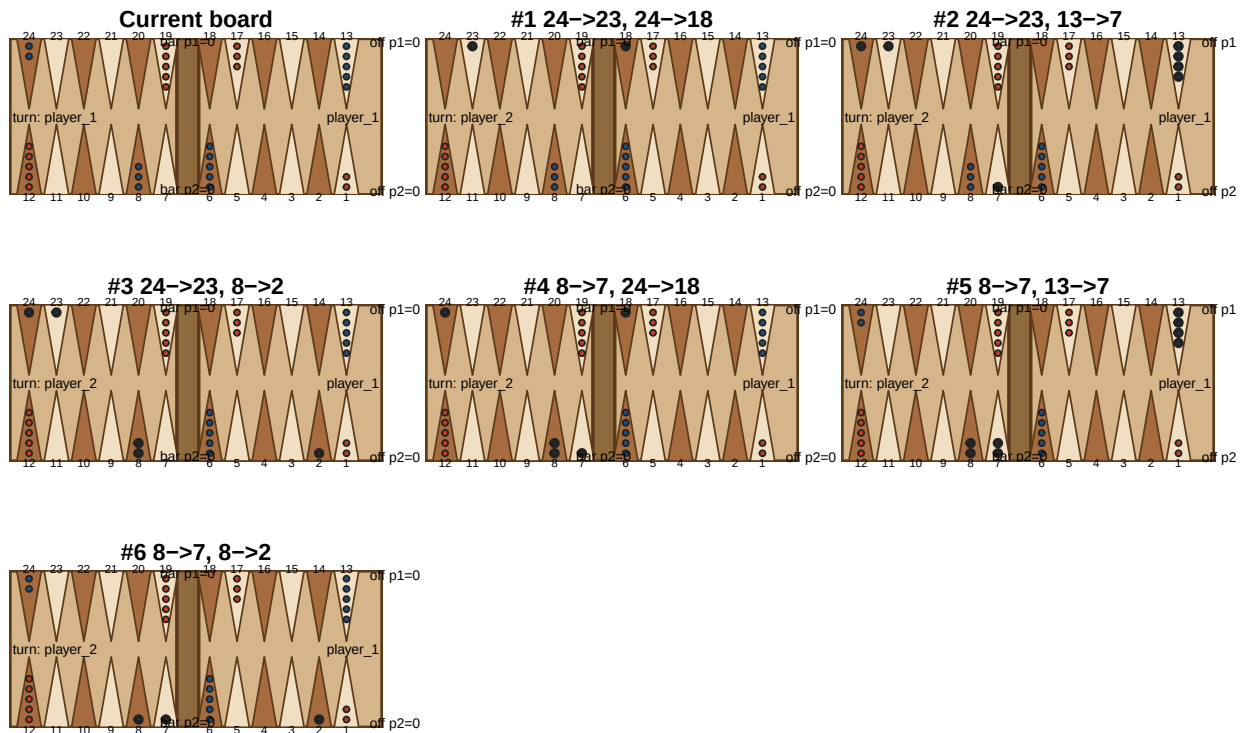
```
legal <- generate_legal_moves(board, roll)

legal_view <- summarize_legal_moves(
  legal,
  max_candidates = 10L,
  print_table = FALSE
)

legal_view[, c("candidate_index", "move_label", "n_steps")]
#>      candidate_index      move_label n_steps
#> 1           1 24->23, 24->18         2
#> 2           2 24->23, 13->7          2
#> 3           3 24->23, 8->2           2
#> 4           4 8->7, 24->18          2
#> 5           5 8->7, 13->7          2
#> 6           6 8->7, 8->2           2
#> 7           7 6->5, 24->18          2
#> 8           8 6->5, 13->7          2
```

```
#> 9          9      6->5, 8->2      2
#> 10         10 24->18, 24->23      2

# Optional visual move diagnostics
bg_plot_legal_moves(board, roll)
```



Interpretation question: how large and how structured is the action set that Thompson must allocate over?

6. Run one finite-budget Thompson evaluation

```
th <- evaluate_actions_thompson(
  board = board,
  roll = roll,
  total_budget = finite_budget,
  rollout_policy = rollout_policy,
  max_rollout_turns = max_rollout_turns,
  fast_diagnostics = FALSE,
  seed = seed
)

print(th)
#> <bg_action_evaluation>
#> method:      thompson
#> candidates:  10
#> total_budget: 1200
```

```

#> recommended: 13->7, 7->6
#> runtime: 0.753 seconds
#> rank action_id action recommended alloc_n estimate uncertainty_sd
#> 1 15 13->7, 7->6 TRUE 392 0.5558376 0.02500036
#> 2 3 24->23, 8->2 FALSE 236 0.5525210 0.03216339
#> 3 5 8->7, 13->7 FALSE 189 0.5183246 0.03606015
#> 4 2 24->23, 13->7 FALSE 78 0.4875000 0.05553819
#> 5 4 8->7, 24->18 FALSE 101 0.4514563 0.04879741
#> 6 1 24->23, 24->18 FALSE 55 0.4385965 0.06515626
#> 7 7 6->5, 24->18 FALSE 44 0.4347826 0.07230943
#> 8 6 8->7, 8->2 FALSE 46 0.4270833 0.07066494
#> 9 9 6->5, 8->2 FALSE 29 0.4193548 0.08723108
#> 10 8 6->5, 13->7 FALSE 30 0.3437500 0.08267973
#> prob_best exp_regret ci95_low ci95_high
#> 0.351562500 0.02466792 0.5068369 0.6048383
#> 0.343750000 0.02622204 0.4894808 0.6155612
#> 0.087890625 0.06147626 0.4476467 0.5890025
#> 0.074218750 0.08968568 0.3786451 0.5963549
#> 0.011718750 0.12805987 0.3558134 0.5470992
#> 0.015625000 0.14552373 0.3108902 0.5663028
#> 0.031250000 0.14313750 0.2930561 0.5765091
#> 0.021484375 0.15629030 0.2885800 0.5655866
#> 0.060546875 0.15645968 0.2483819 0.5903278
#> 0.001953125 0.23841505 0.1816977 0.5058023
summary(th)
#> method total_budget n_candidates recommended_index recommended_move_label
#> 1 thompson 1200 10 15 13->7, 7->6
#> recommended_estimate recommended_prob_best recommended_expected_regret
#> 1 0.5558376 0.3515625 0.02466792
#> recommended_allocation_count runtime_seconds
#> 1 392 0.753
compare_action_posteriors(th, top_n = 8L)[, c(
  "move_label", "recommended", "allocation_count",
  "estimate", "uncertainty_sd", "prob_best", "exp_regret"
)]
#> move_label recommended allocation_count estimate uncertainty_sd
#> 1 13->7, 7->6 TRUE 392 0.5558376 0.02500036
#> 2 24->23, 8->2 FALSE 236 0.5525210 0.03216339
#> 3 8->7, 13->7 FALSE 189 0.5183246 0.03606015
#> 4 24->23, 13->7 FALSE 78 0.4875000 0.05553819
#> 5 8->7, 24->18 FALSE 101 0.4514563 0.04879741
#> 6 24->23, 24->18 FALSE 55 0.4385965 0.06515626
#> 7 6->5, 24->18 FALSE 44 0.4347826 0.07230943
#> 8 8->7, 8->2 FALSE 46 0.4270833 0.07066494
#> prob_best exp_regret
#> 1 0.35156250 0.02466792
#> 2 0.34375000 0.02622204
#> 3 0.08789062 0.06147626
#> 4 0.07421875 0.08968568
#> 5 0.01171875 0.12805987
#> 6 0.01562500 0.14552373
#> 7 0.03125000 0.14313750
#> 8 0.02148438 0.15629030

```

```
explain_move_evaluation(th)
```

```
#> [1] "Method thompson selected candidate 15 (`13->7, 7->6`) with posterior mean 0.556 after 392 rollo"
```

Interpretation:

- `allocation_count`: where finite budget went,
- `estimate + uncertainty_sd`: current value and uncertainty,
- `prob_best`: posterior plausibility that action is best,
- `exp_regret`: posterior downside if that action were chosen.

7. Compare Thompson against baseline methods at same budget

This gives a same-instance, same-budget cross-method snapshot.

```
methods <- compare_methods_on_position(
  board = board,
  roll = roll,
  methods = c("thompson", "equal", "ucb", "ocba", "greedy", "ttts"),
  total_budget = finite_budget,
  rollout_policy = rollout_policy,
  max_rollout_turns = max_rollout_turns,
  fast_diagnostics = FALSE,
  seed = seed + 1L
)

methods$summary[, c(
  "method",
  "recommended_move_label",
  "recommended_estimate",
  "recommended_prob_best",
  "recommended_expected_regret",
  "recommended_allocation_count",
  "estimate_gap_top2",
  "runtime_seconds"
)]
#>      method recommended_move_label recommended_estimate recommended_prob_best
#> 1 thompson           8->7, 13->7           0.5892070           0.6835938
#> 2  equal           24->23, 24->18           0.6229508           0.7968750
#> 3   ucb           13->7, 7->6           0.5738397           0.5058594
#> 4  ocba           13->7, 7->6           0.5677966           0.5859375
#> 5 greedy           13->7, 7->6           0.5797414           0.1308594
#> 6  ttts           13->7, 7->6           0.5700326           0.4843750
#>      recommended_expected_regret recommended_allocation_count estimate_gap_top2
#> 1           0.007937240                452           0.04323004
#> 2           0.007354964                120           0.07377049
#> 3           0.018431475                235           0.03098252
#> 4           0.014171189                293           0.02977380
#> 5           0.142172788               1158           0.07974138
#> 6           0.018078745                305           0.02025882
#>      runtime_seconds
#> 1           0.762
```

```
#> 2          0.771
#> 3          0.801
#> 4          0.778
#> 5          0.725
#> 6          0.768
```

Interpretation: this is not a final benchmark, but it is a clean first check of whether Thompson is competitive on this specific state.

8. Trace Thompson allocation dynamics over time

```
tr <- trace_thompson_allocation(
  board = board,
  roll = roll,
  method = "thompson",
  total_budget = finite_budget,
  trace_every = trace_every,
  rollout_policy = rollout_policy,
  max_rollout_turns = max_rollout_turns,
  seed = seed + 2L
)

tr$checkpoint_summary[, c(
  "checkpoint",
  "selected_move_label",
  "leader_move_label",
  "leader_estimate",
  "leader_posterior_sd",
  "leader_allocation_count"
)]

#>   checkpoint selected_move_label leader_move_label leader_estimate
#> 1         120          8->7, 8->2          8->7, 8->2      0.6153846
#> 2         240          6->5, 24->18          8->7, 8->2      0.5862069
#> 3         360          24->23, 8->2          6->5, 24->18      0.5581395
#> 4         480          24->23, 24->18          24->23, 13->7      0.5324675
#> 5         600          24->23, 13->7          24->23, 13->7      0.5288462
#> 6         720          24->23, 13->7          24->23, 13->7      0.5263158
#> 7         840          8->7, 8->2          24->23, 13->7      0.5384615
#> 8         960          24->23, 13->7          24->23, 13->7      0.5511111
#> 9        1080          6->5, 13->7          24->23, 13->7      0.5508772
#> 10        1200          8->7, 8->2          24->23, 13->7      0.5429799
#>   leader_posterior_sd leader_allocation_count
#> 1          0.07692308              37
#> 2          0.05250199              85
#> 3          0.07486652              41
#> 4          0.05649437              75
#> 5          0.04871373             102
#> 6          0.04313356             131
#> 7          0.03823462             167
#> 8          0.03308528             223
#> 9          0.02941216             283
```

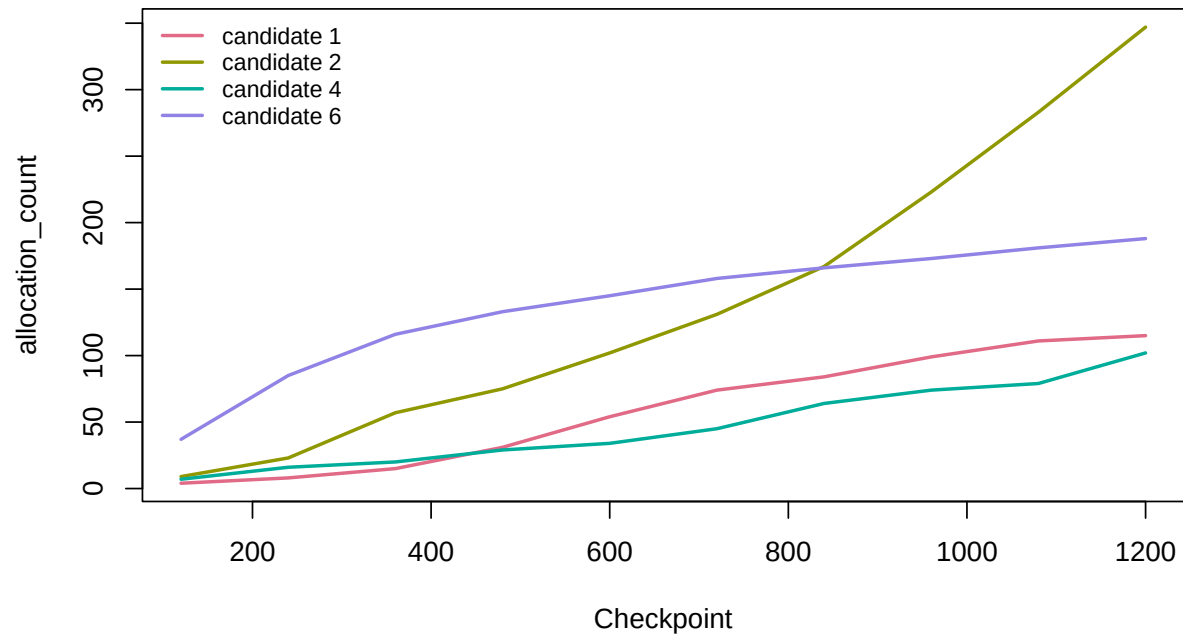
```
#> 10
```

```
0.02662720
```

```
347
```

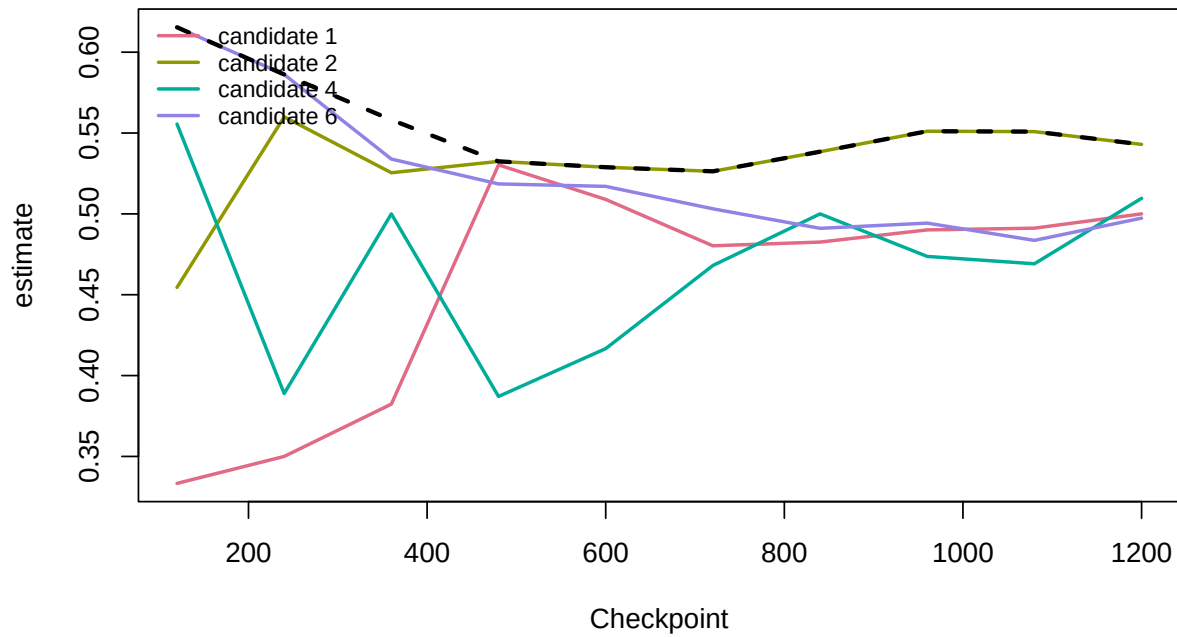
```
plot_thompson_convergence(tr, metric = "allocation_count")
```

Thompson convergence: thompson



```
plot_thompson_convergence(tr, metric = "estimate")
```

Thompson convergence: thompson



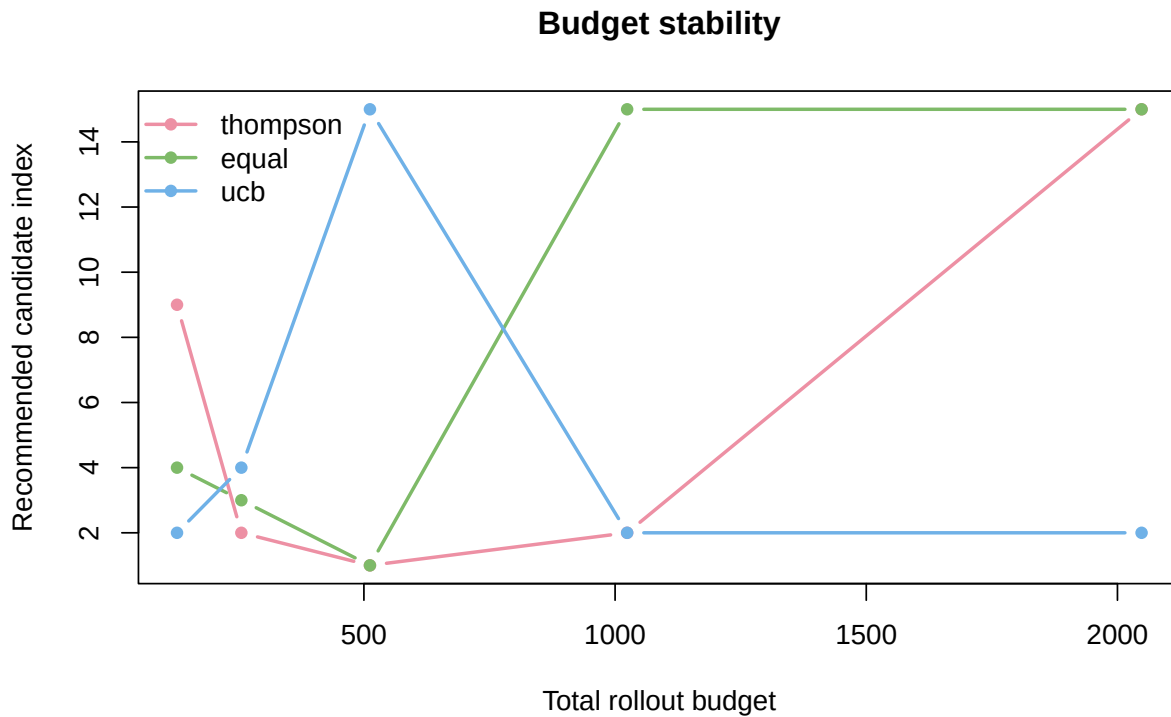
Interpretation questions:

- does allocation concentrate too early?
- do challengers keep some budget in hard states?
- does the leader stabilize as evidence accumulates?

9. Plot recommendation stability across budgets

This is an explicit instability/mistake diagnostic.

```
bg_plot_budget_stability(  
  board = board,  
  roll = roll,  
  budgets = budget_grid,  
  methods = c("thompson", "equal", "ucb"),  
  rollout_policy = rollout_policy,  
  max_rollout_turns = max_rollout_turns,  
  seed = seed + 3L  
)
```



Interpretation:

- low-budget flips are expected in hard states,
- what matters is whether instability shrinks as budget grows.

10. Build a high-budget reference estimate and certify it

```
ref <- approximate_action_reference(
  board = board,
  roll = roll,
  truth_budget = reference_budget,
  rollout_policy = rollout_policy,
  max_rollout_turns = max_rollout_turns,
  seed = seed
)

cert <- certify_reference_truth(reference = ref)
cert$certificate
#>   n_actions reference_best_index reference_best_move_label
#> 1      10                2      24->23, 13->7
#>   reference_second_index reference_second_move_label top_two_gap_estimate
#> 1                3      24->23, 8->2      0.01164725
#>   top_two_gap_se top_two_gap_lower_95 top_two_gap_upper_95 certified
#> 1      0.0203369      -0.02821307      0.05150758      FALSE
#>   difficulty_score difficulty_label
#> 1      85.85714      hard
```

Important interpretation rule: reference is a proxy truth. Certification helps flag when top-two separation is weak.

11. Compare finite Thompson output to the reference

```
cmp <- compare_thompson_to_reference(
  board = board,
  roll = roll,
  method = "thompson",
  total_budget = finite_budget,
  reference = ref,
  reference_certificate = cert,
  rollout_policy = rollout_policy,
  max_rollout_turns = max_rollout_turns,
  seed = seed + 4L
)

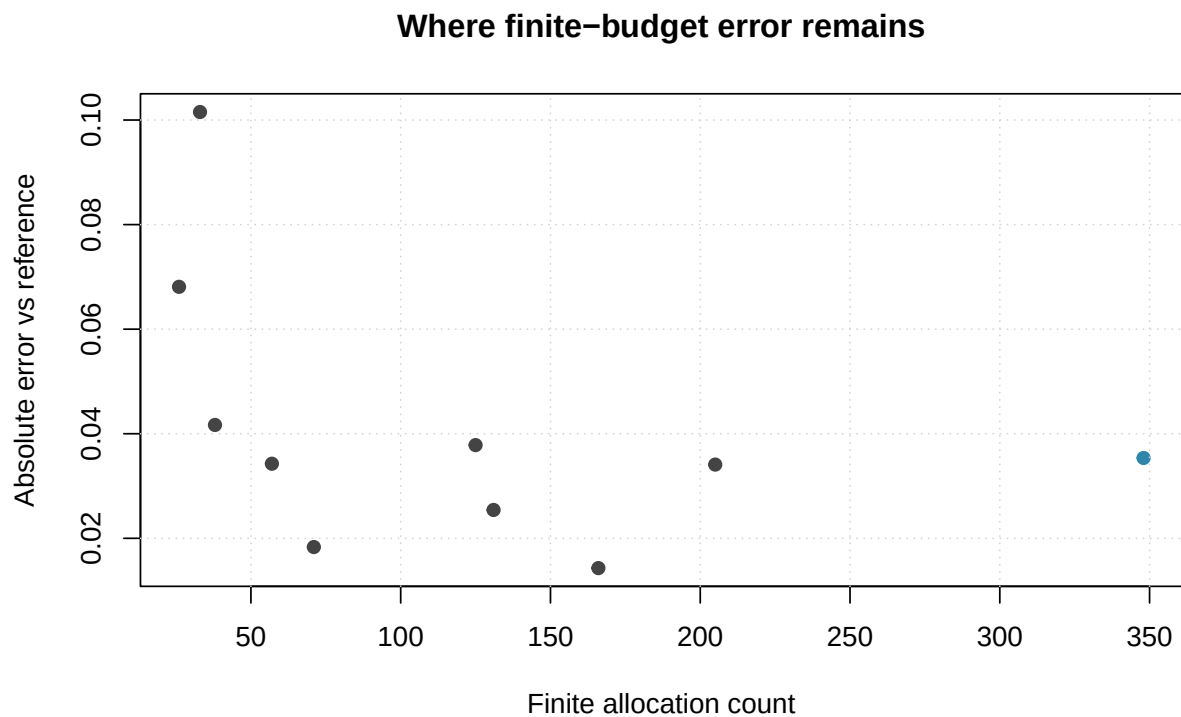
cmp$summary[, c(
  "method", "total_budget", "reference_budget",
  "proxy_pcs", "simple_regret", "mse",
  "finite_runtime_seconds", "reference_runtime_seconds",
  "difficulty_label", "reference_certified"
)]
#>      method total_budget reference_budget proxy_pcs simple_regret      mse
#> 1 thompson          1200          12000         1         0 0.0022889
#>      finite_runtime_seconds reference_runtime_seconds difficulty_label
#> 1              0.749              7.585              hard
#>      reference_certified
#> 1              FALSE

cmp_actions <- cmp$action_table[order(-cmp$action_table$finite_allocation_count), c(
  "move_label", "finite_recommended", "reference_best",
  "finite_allocation_count", "finite_estimate",
  "reference_estimate", "abs_error"
)]

head(cmp_actions, 10L)
#>      move_label finite_recommended reference_best finite_allocation_count
#> 2  24->23, 13->7              TRUE              TRUE              348
#> 3  24->23, 8->2              FALSE              FALSE              205
#> 5   8->7, 13->7              FALSE              FALSE              166
#> 4   8->7, 24->18             FALSE              FALSE              131
#> 8   6->5, 13->7              FALSE              FALSE              125
#> 9   6->5, 8->2              FALSE              FALSE              71
#> 10  13->7, 7->6              FALSE              FALSE              57
#> 1  24->23, 24->18             FALSE              FALSE              38
#> 7   6->5, 24->18             FALSE              FALSE              33
#> 6   8->7, 8->2              FALSE              FALSE              26
#>      finite_estimate reference_estimate abs_error
#> 2      0.5757143      0.5403494 0.03536487
#> 3      0.5628019      0.5287022 0.03409977
```

```
#> 5      0.5119048      0.5262063 0.01430156
#> 4      0.5112782      0.4858569 0.02542129
#> 8      0.5078740      0.4700499 0.03782410
#> 9      0.4863014      0.4679700 0.01833132
#> 10     0.4915254      0.5257903 0.03426493
#> 1      0.4500000      0.4916805 0.04168053
#> 7      0.3714286      0.4729617 0.10153316
#> 6      0.4285714      0.4966722 0.06810078
```

```
plot(
  cmp_actions$finite_allocation_count,
  cmp_actions$abs_error,
  pch = 19,
  col = ifelse(cmp_actions$finite_recommended, "#2E86AB", "#444444"),
  xlab = "Finite allocation count",
  ylab = "Absolute error vs reference",
  main = "Where finite-budget error remains"
)
grid()
```



```
# Richer finite-vs-reference diagnostics
cmp_diag <- cmp$action_table[, c(
  "move_label", "finite_recommended", "reference_best",
  "finite_allocation_count", "finite_estimate", "finite_posterior_sd",
  "reference_estimate", "abs_error"
)]
```

```

old_par <- par(no.readonly = TRUE)
par(mfrow = c(2, 2), mar = c(4, 4, 3, 1))

# (1) Calibration-style scatter: finite estimate vs reference estimate
plot(
  cmp_diag$reference_estimate,
  cmp_diag$finite_estimate,
  pch = 19,
  col = ifelse(cmp_diag$finite_recommended, "#2E86AB", "#444444"),
  xlab = "Reference estimate",
  ylab = "Finite Thompson estimate",
  main = "Finite vs reference (action-level)"
)
abline(0, 1, lty = 2, col = "red")
text(
  cmp_diag$reference_estimate,
  cmp_diag$finite_estimate,
  labels = seq_len(nrow(cmp_diag)),
  pos = 3,
  cex = 0.7
)

# (2) Sorted absolute errors by action
ord_err <- order(cmp_diag$abs_error, decreasing = TRUE)
err_sorted <- cmp_diag[ord_err, , drop = FALSE]
plot(
  err_sorted$abs_error,
  seq_len(nrow(err_sorted)),
  pch = 19,
  yaxt = "n",
  xlab = "Absolute error",
  ylab = "Action",
  main = "Largest finite-vs-reference errors"
)
segments(0, seq_len(nrow(err_sorted)), err_sorted$abs_error, seq_len(nrow(err_sorted)))
axis(2, at = seq_len(nrow(err_sorted)), labels = err_sorted$move_label, las = 1, cex.axis = 0.65)

# (3) Posterior uncertainty vs absolute error
plot(
  cmp_diag$finite_posterior_sd,
  cmp_diag$abs_error,
  pch = 19,
  cex = pmax(0.7, sqrt(cmp_diag$finite_allocation_count / max(cmp_diag$finite_allocation_count))),
  col = ifelse(cmp_diag$reference_best, "#C0392B", "#444444"),
  xlab = "Finite posterior SD",
  ylab = "Absolute error vs reference",
  main = "Uncertainty-error relationship"
)
legend(
  "topright",
  legend = c("Reference-best action", "Other actions"),
  col = c("#C0392B", "#444444"),
  pch = 19,

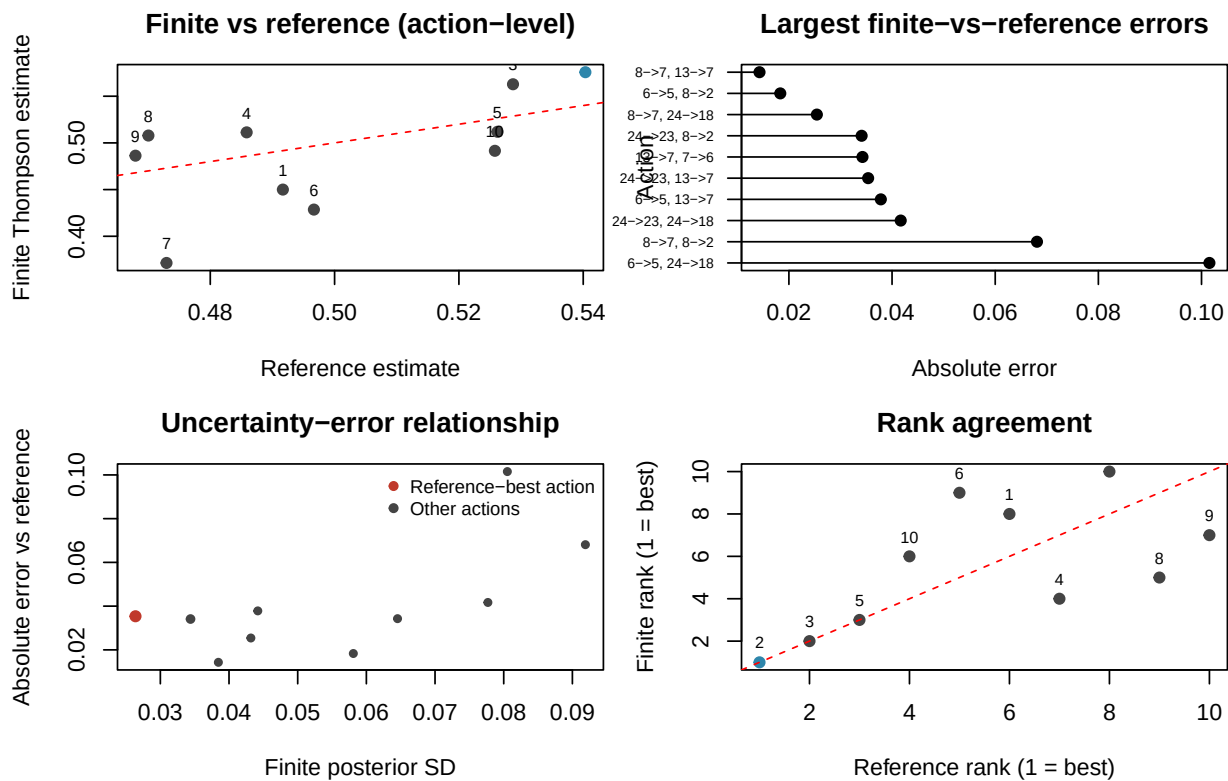
```

```

bty = "n",
cex = 0.8
)

# (4) Rank agreement: finite ranking vs reference ranking
rank_df <- cmp_diag
rank_df$finite_rank <- rank(-rank_df$finite_estimate, ties.method = "first")
rank_df$reference_rank <- rank(-rank_df$reference_estimate, ties.method = "first")
plot(
  rank_df$reference_rank,
  rank_df$finite_rank,
  pch = 19,
  col = ifelse(rank_df$finite_recommended, "#2E86AB", "#444444"),
  xlab = "Reference rank (1 = best)",
  ylab = "Finite rank (1 = best)",
  main = "Rank agreement"
)
abline(0, 1, lty = 2, col = "red")
text(rank_df$reference_rank, rank_df$finite_rank, labels = seq_len(nrow(rank_df)), pos = 3, cex = 0.7)

```



```
par(old_par)
```

Interpretation:

- `proxy_pcs`: did finite recommendation match reference-best?
- `simple_regret`: decision loss relative to reference-best value,

- mse: whole action-vector estimation error,
- error-vs-allocation plot: where finite information is still weak,
- calibration scatter: whether finite action estimates track reference values,
- sorted-error chart: which specific actions are hardest at this budget,
- uncertainty-error plot: whether high posterior uncertainty aligns with larger reference gaps,
- rank-agreement plot: whether finite ordering is close to reference ordering.

12. Study budget tradeoff directly

```
trade <- study_budget_tradeoff(
  board = board,
  roll = roll,
  method = "thompson",
  budgets = budget_grid,
  truth_budget = reference_budget,
  rollout_policy = rollout_policy,
  max_rollout_turns = max_rollout_turns,
  fast_diagnostics = FALSE,
  seed = seed + 5L
)

trade_view <- trade$results[, c(
  "total_budget",
  "chosen_move_label",
  "truth_best_move_label",
  "correct_selection",
  "simple_regret",
  "mse",
  "runtime_seconds"
)]

trade_view
#>   total_budget chosen_move_label truth_best_move_label correct_selection
#> 1         128    24->23, 24->18      24->23, 13->7             FALSE
#> 2         256      6->5, 13->7      24->23, 13->7             FALSE
#> 3         512    24->23, 13->7      24->23, 13->7              TRUE
#> 4        1024    24->23, 13->7      24->23, 13->7              TRUE
#> 5        2048    24->23, 8->2      24->23, 13->7             FALSE
#>   simple_regret      mse runtime_seconds
#> 1  0.072795341 0.018478694      0.085
#> 2  0.060732113 0.017684382      0.164
#> 3  0.000000000 0.023321068      0.312
#> 4  0.000000000 0.007617595      0.660
#> 5  0.007071547 0.014792810      1.294

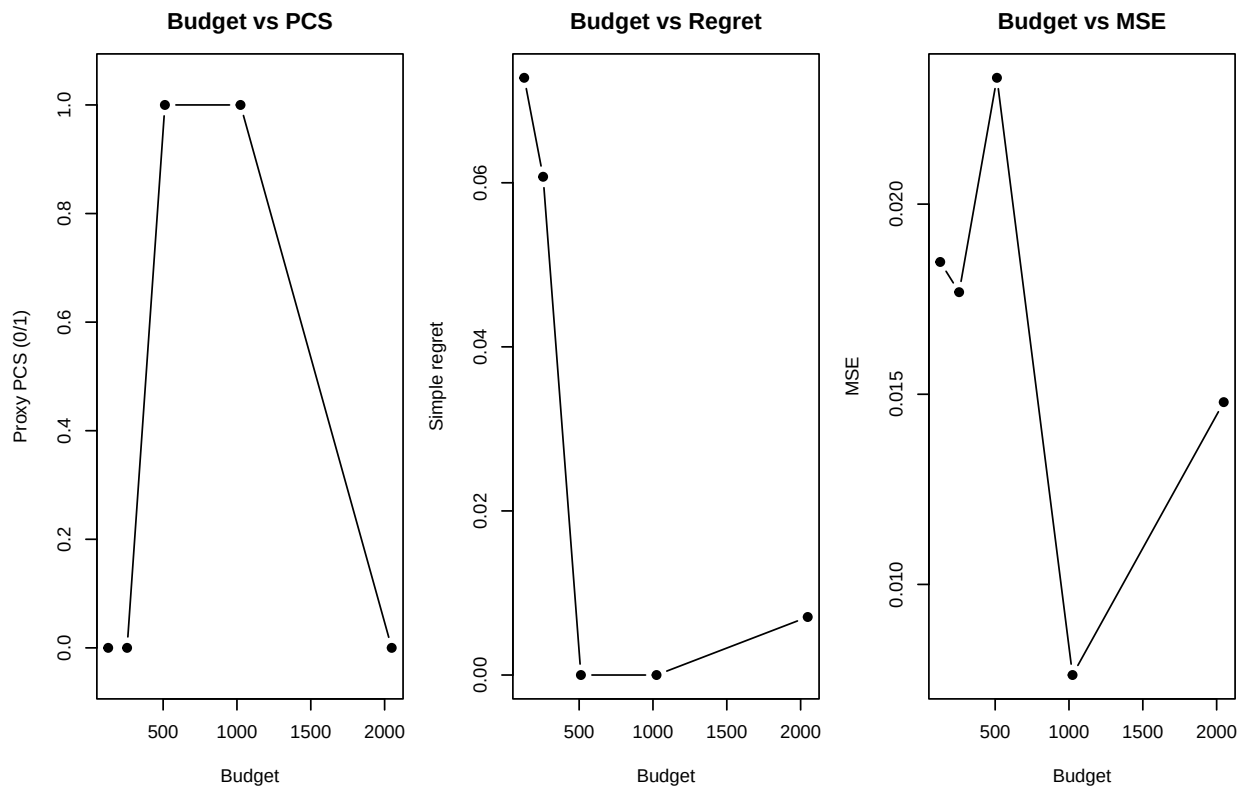
old_par <- par(no.readonly = TRUE)
par(mfrow = c(1, 3), mar = c(4, 4, 3, 1))

plot(
  trade_view$total_budget,
  as.numeric(trade_view$correct_selection),
```

```

type = "b", pch = 19, ylim = c(-0.05, 1.05),
xlab = "Budget", ylab = "Proxy PCS (0/1)", main = "Budget vs PCS"
)
plot(
trade_view$total_budget,
trade_view$simple_regret,
type = "b", pch = 19,
xlab = "Budget", ylab = "Simple regret", main = "Budget vs Regret"
)
plot(
trade_view$total_budget,
trade_view$mse,
type = "b", pch = 19,
xlab = "Budget", ylab = "MSE", main = "Budget vs MSE"
)

```



```

par(old_par)

if (all(c("chosen_estimate", "chosen_truth_value", "truth_best_value") %in% names(trade$results))) {
gap_df <- trade$results[order(trade$results$total_budget), c(
"total_budget", "chosen_estimate", "chosen_truth_value", "truth_best_value"
)]
gap_df$finite_ref_gap <- gap_df$chosen_estimate - gap_df$chosen_truth_value
gap_df$to_ref_best_gap <- gap_df$truth_best_value - gap_df$chosen_truth_value

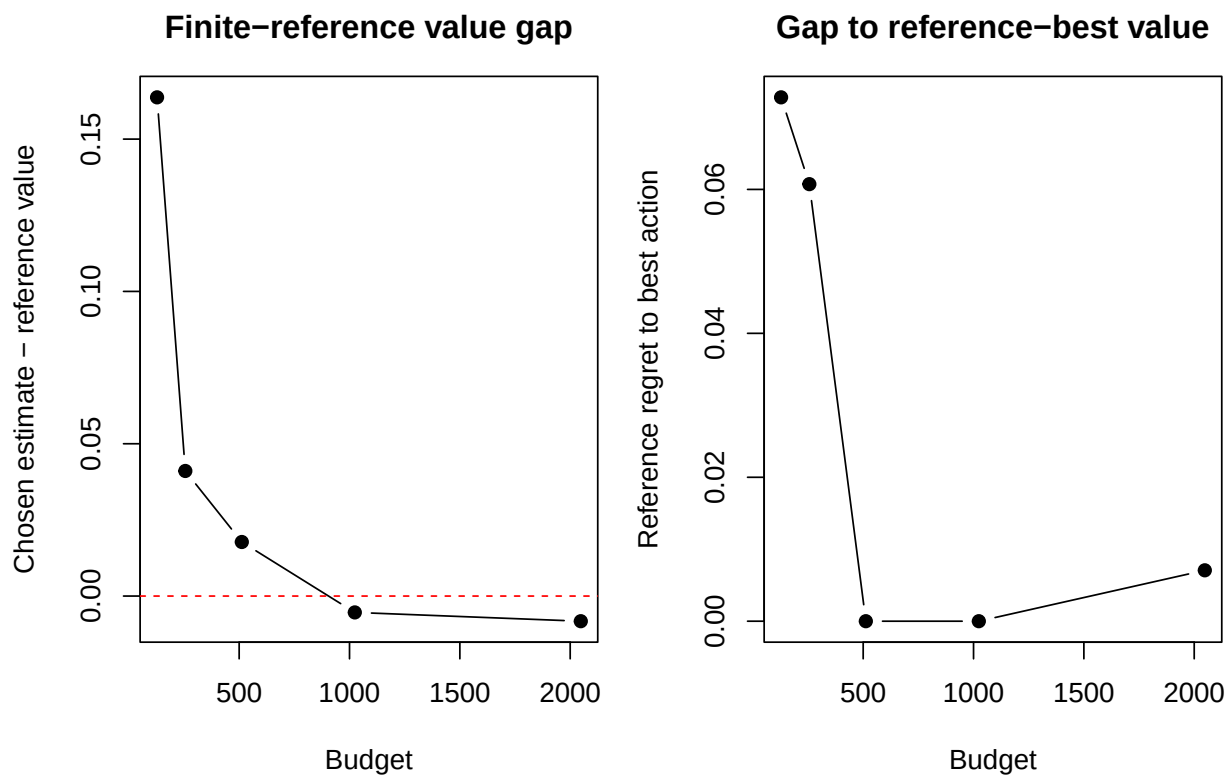
old_par <- par(no.readonly = TRUE)
par(mfrow = c(1, 2), mar = c(4, 4, 3, 1))

```

```

plot(
  gap_df$total_budget,
  gap_df$finite_ref_gap,
  type = "b", pch = 19,
  xlab = "Budget",
  ylab = "Chosen estimate - reference value",
  main = "Finite-reference value gap"
)
abline(h = 0, lty = 2, col = "red")
plot(
  gap_df$total_budget,
  gap_df$to_ref_best_gap,
  type = "b", pch = 19,
  xlab = "Budget",
  ylab = "Reference regret to best action",
  main = "Gap to reference-best value"
)
par(old_par)
}

```



Interpretation:

- this is the key convergence story for one instance,
- finite-budget non-monotonicity can happen, focus on trend,
- include runtime when discussing quality gains,
- the gap plots show whether chosen finite values and chosen actions approach the reference trajectory as budget grows.

13. Check variance-control sensitivity

```
var_study <- study_variance_controls(  
  board = board,  
  roll = roll,  
  method = "thompson",  
  total_budget = finite_budget,  
  dice_modes = c("iid", "stratified_first_roll", "stratified_first_two_rolls"),  
  crn_values = c(FALSE, TRUE),  
  truth_budget = reference_budget,  
  rollout_policy = rollout_policy,  
  max_rollout_turns = max_rollout_turns,  
  fast_diagnostics = FALSE,  
  seed = seed + 6L  
)  
  
var_study$results[, c(  
  "dice_mode", "crn", "chosen_move_label",  
  "correct_selection", "simple_regret", "mse", "runtime_seconds"  
)]  
  
#>           dice_mode  crn chosen_move_label correct_selection  
#> 1             iid FALSE      13->7, 7->6             FALSE  
#> 2             iid  TRUE      24->23, 13->7             FALSE  
#> 3 stratified_first_roll FALSE      24->23, 13->7             FALSE  
#> 4 stratified_first_roll  TRUE      13->7, 7->6             FALSE  
#> 5 stratified_first_two_rolls FALSE      13->7, 7->6             FALSE  
#> 6 stratified_first_two_rolls  TRUE      13->7, 7->6             FALSE  
#>  simple_regret      mse runtime_seconds  
#> 1  0.002079867 0.008331822          0.742  
#> 2  0.007903494 0.002366913          0.820  
#> 3  0.007903494 0.012372190          0.744  
#> 4  0.002079867 0.006220574          0.754  
#> 5  0.002079867 0.008615714          0.744  
#> 6  0.002079867 0.009283735          0.762
```

Interpretation: if conclusions change radically across settings, report that explicitly.

14. Run multi-case Thompson benchmark and summarize

To use `summarize_thompson_benchmark()`, first run `benchmark_thompson()`.

```
cases <- list(  
  bg_benchmark_case(bg_initial_board(), bg_roll(1L, 6L), case_id = "init_1_6"),  
  bg_benchmark_case(bg_initial_board(), bg_roll(3L, 2L), case_id = "init_3_2"),  
  bg_benchmark_case(bg_initial_board(), bg_roll(5L, 4L), case_id = "init_5_4"),  
  bg_benchmark_case(bg_initial_board(), bg_roll(2L, 1L), case_id = "init_2_1"),  
  bg_benchmark_case(bg_initial_board(), bg_roll(6L, 6L), case_id = "init_6_6"),  
  bg_benchmark_case(bg_initial_board(), bg_roll(4L, 3L), case_id = "init_4_3")  
)  
  
if (isTRUE(getOption("bg_quick_mode", TRUE))) {
```

```

cases <- cases[1:3]
}

bm <- benchmark_thompson(
  cases = cases,
  budgets = benchmark_budgets,
  baselines = c("equal", "ucb", "ocba", "greedy"),
  include_ttts = TRUE,
  reference_budget = reference_budget,
  rollout_policy = rollout_policy,
  max_rollout_turns = max_rollout_turns,
  seed = seed + 7L
)

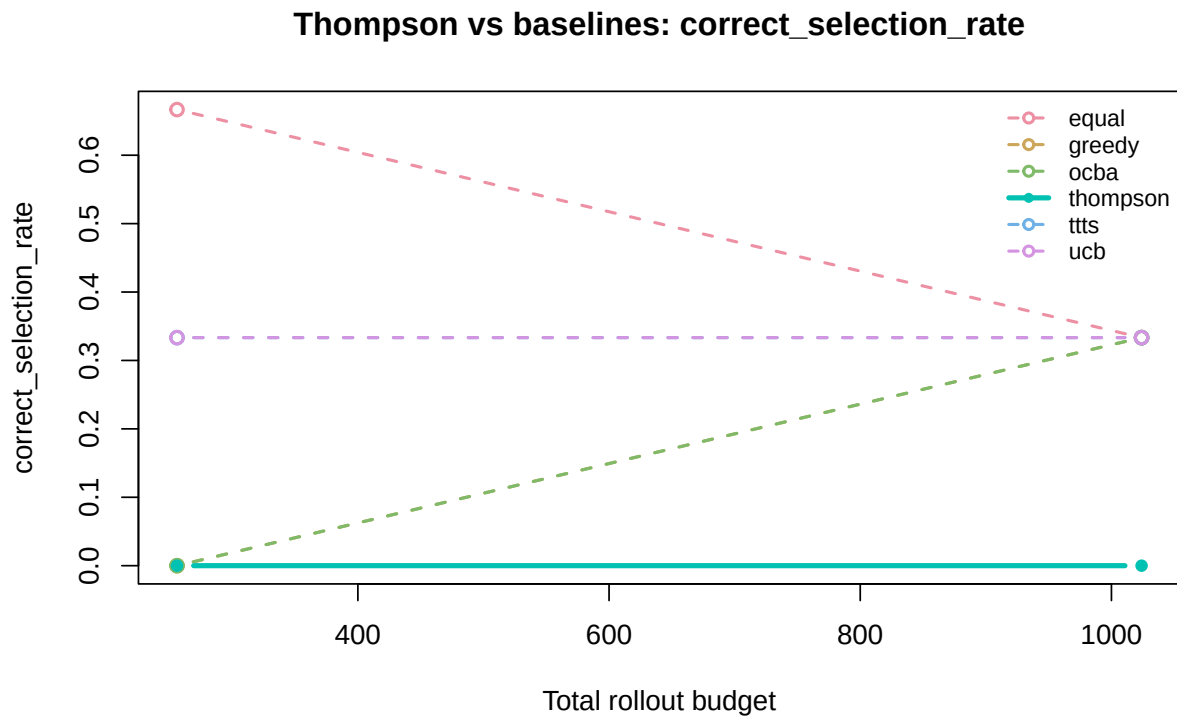
focus <- summarize_thompson_benchmark(bm)

head(focus$thompson, 12L)
#>      method total_budget dice_mode   crn total_cases multi_move_cases
#> 1 thompson          256      iid FALSE         3             3
#> 7 thompson         1024      iid FALSE         3             3
#>   correct_selection_count correct_selection_rate reference_best_move_label
#> 1                      0                      0                <varies>
#> 7                      0                      0                <varies>
#>   most_selected_move_label mean_n_legal_moves probability_correct_selection
#> 1          13->11, 11->8                24                      0
#> 7          24->20, 13->8                24                      0
#>   mean_simple_regret   mean_mse mean_runtime_seconds n_cases decision_cases
#> 1      0.03332303 0.021018915      0.1646667         3             3
#> 7      0.04281801 0.009902523      0.6826667         3             3
head(focus$relative_to_thompson, 20L)
#>   baseline_method total_budget dice_mode   crn thompson_advantage_pcs
#> 1      equal          256      iid FALSE      -0.6666667
#> 2      ucb          256      iid FALSE      -0.3333333
#> 3      ocba          256      iid FALSE       0.0000000
#> 4      greedy         256      iid FALSE       0.0000000
#> 5      ttts          256      iid FALSE      -0.3333333
#> 6      equal         1024      iid FALSE      -0.3333333
#> 7      ucb          1024      iid FALSE      -0.3333333
#> 8      ocba          1024      iid FALSE      -0.3333333
#> 9      greedy         1024      iid FALSE      -0.3333333
#> 10     ttts          1024      iid FALSE      -0.3333333
#>   thompson_advantage_regret thompson_advantage_mse thompson_runtime_ratio
#> 1      -0.0114303746      -0.013260715      0.9821074
#> 2      -0.0126461763      -0.001633566      0.8758865
#> 3       0.0044178566      -0.003736644      0.9592233
#> 4       0.0001131969      -0.007557563      1.0020284
#> 5       0.0053119390      -0.001145595      0.9724409
#> 6      -0.0237216478      -0.006197640      1.0353893
#> 7      -0.0113327837      -0.006955204      1.0054001
#> 8      -0.0246810871      -0.003065141      0.9784998
#> 9      -0.0221835497       0.007081597      0.8726033
#> 10     -0.0224380567      -0.003692948      0.7825755
head(focus$by_difficulty, 20L)

```

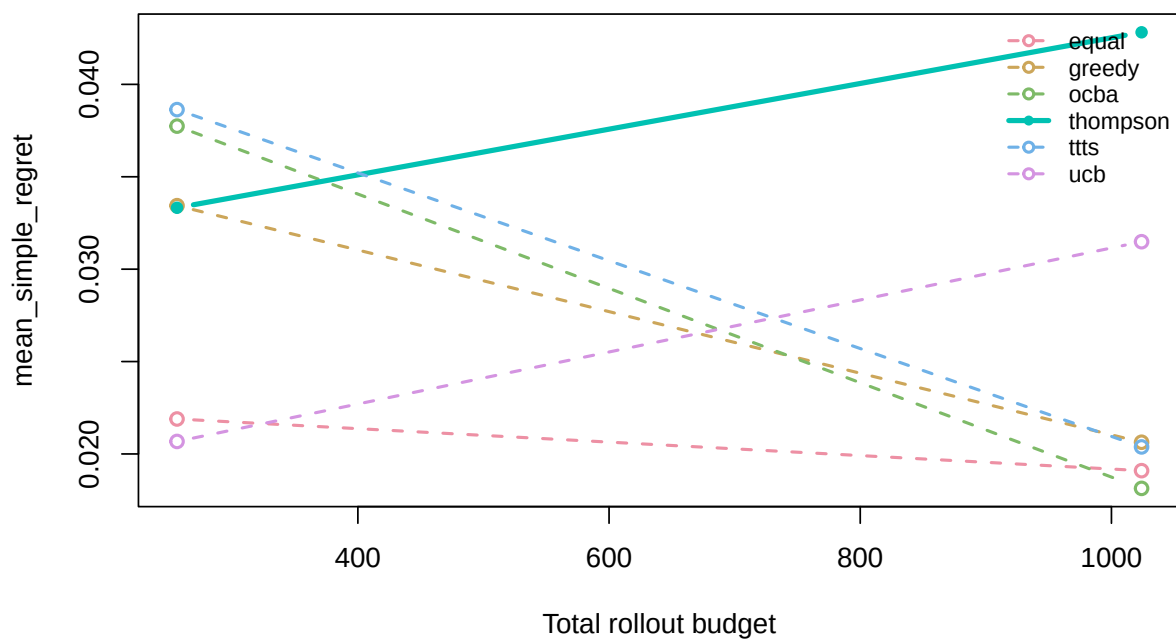
```
#>      method difficulty_label n_cases proxy_pcs mean_simple_regret mean_mse
#> 1    equal                hard      6 0.5000000      0.02049451 0.005731541
#> 2   greedy                hard      6 0.1666667      0.02703534 0.015222736
#> 3    ocba                hard      6 0.1666667      0.02793890 0.012059826
#> 4 thompson                hard      6 0.0000000      0.03807052 0.015460719
#> 5     ttts                hard      6 0.3333333      0.02950746 0.013041448
#> 6      ucb                hard      6 0.3333333      0.02608104 0.011166334
#> mean_runtime_seconds
#> 1          0.4135000
#> 2          0.4733333
#> 3          0.4346667
#> 4          0.4236667
#> 5          0.5208333
#> 6          0.4335000
```

```
plot_thompson_vs_baselines(bm, metric = "correct_selection_rate")
```



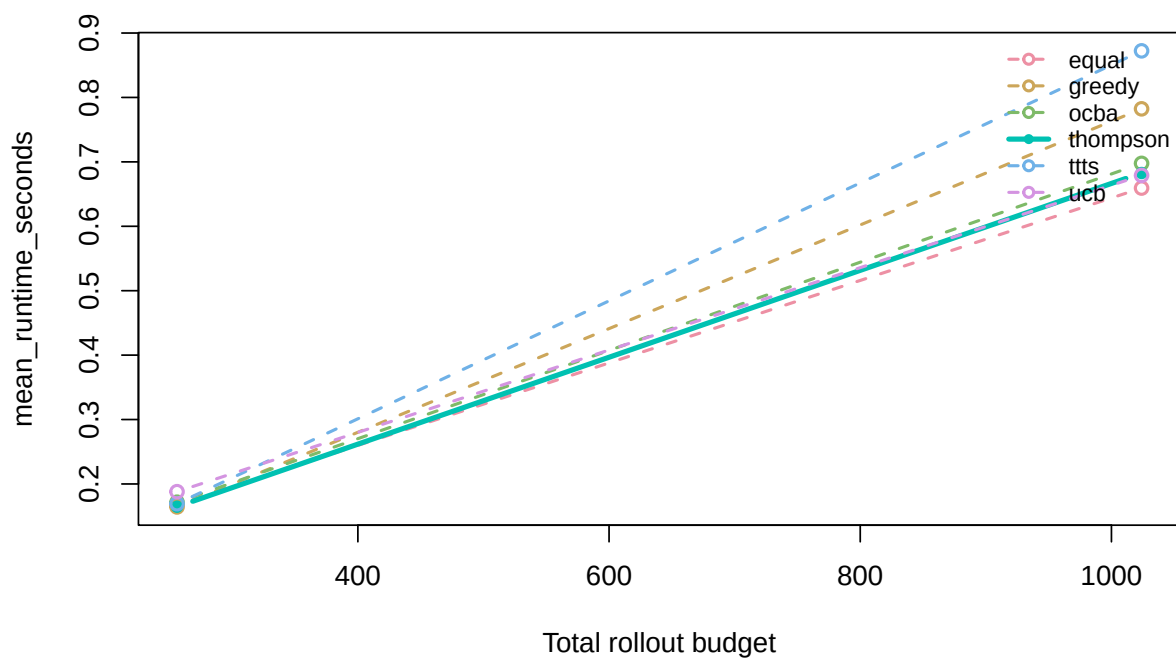
```
plot_thompson_vs_baselines(bm, metric = "mean_simple_regret")
```

Thompson vs baselines: mean_simple_regret



```
plot_thompson_vs_baselines(bm, metric = "mean_runtime_seconds")
```

Thompson vs baselines: mean_runtime_seconds



Interpretation:

- this is where state-dependent strengths and weaknesses become visible,
- report both favorable and unfavorable cells,
- discuss quality and runtime together.

15. Interpretation checklist (what to say in results)

For each figure/table, answer explicitly:

1. what statistical question this output is answering,
2. whether the state appears easy/hard (`difficulty_label`, certification),
3. whether finite Thompson is converging toward reference behavior,
4. where errors remain and why they may remain,
5. whether gains persist against baselines,
6. what runtime cost was paid.

16. What counts as encouraging vs cautionary

Encouraging patterns:

- allocation increasingly concentrates on plausible best actions,
- regret and MSE generally decline with budget,
- proxy PCS improves and recommendation flips decrease,
- Thompson is competitive or stronger than baselines on many cells.

Cautionary patterns:

- persistent instability at moderate budgets,
- large finite-vs-reference gaps despite high finite budget,
- improvements that disappear outside one easy case,
- strong dependence on one variance-control setting,
- reference certification indicates unresolved near-tie ambiguity.

17. Final takeaway

This workflow is not about generating large tables. It is about finite-budget statistical decision evaluation:

- define one concrete decision instance,
- study Thompson allocation behavior,
- compare to a stronger reference estimate,
- check robustness across budgets and variance controls,
- validate broader behavior across multiple benchmark cases.

That is the core research use of this package.